

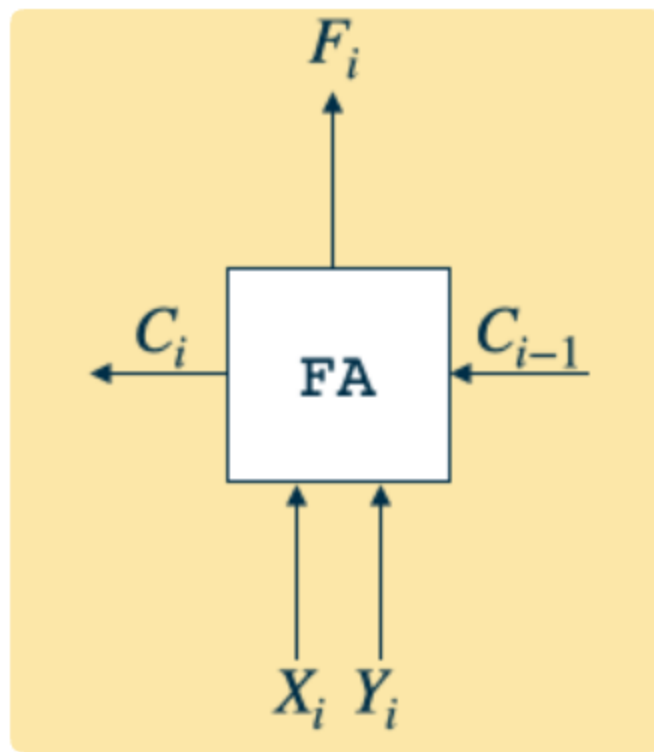
2-2 运算方法和运



1.基本运算部件

1.1串行进位加法器

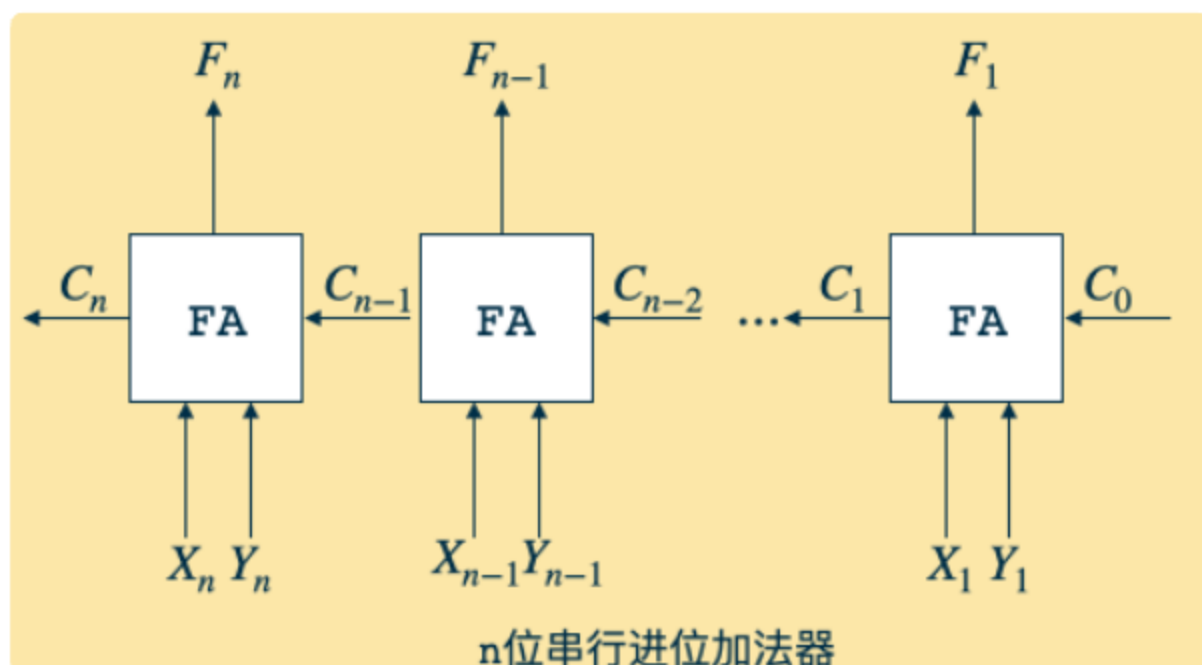
全加器用来实现两个本位数加上低位进位生成一位本位以及一位向高位的进位



运算法则

和: $S_i = X_i \oplus Y_i \oplus C_{i-1}$
进位: $C_i = X_i C_{i-1} + Y_i C_{i-1} + X_i Y_i$

将n个全加器相连得到n位串行进位加法器, X和Y逐位相加, 位间进位串行传送



运算法则

和: $S_i = X_i \oplus Y_i \oplus C_{i-1}$
进位: $C_i = X_i C_{i-1} + Y_i C_{i-1} + X_i Y_i$

每个进位表达式都含有Xi+Yi和XiYi, Xi+Yi为Pi, XiYi为Gi

$C_1 = G_1 + P_1 C_0$

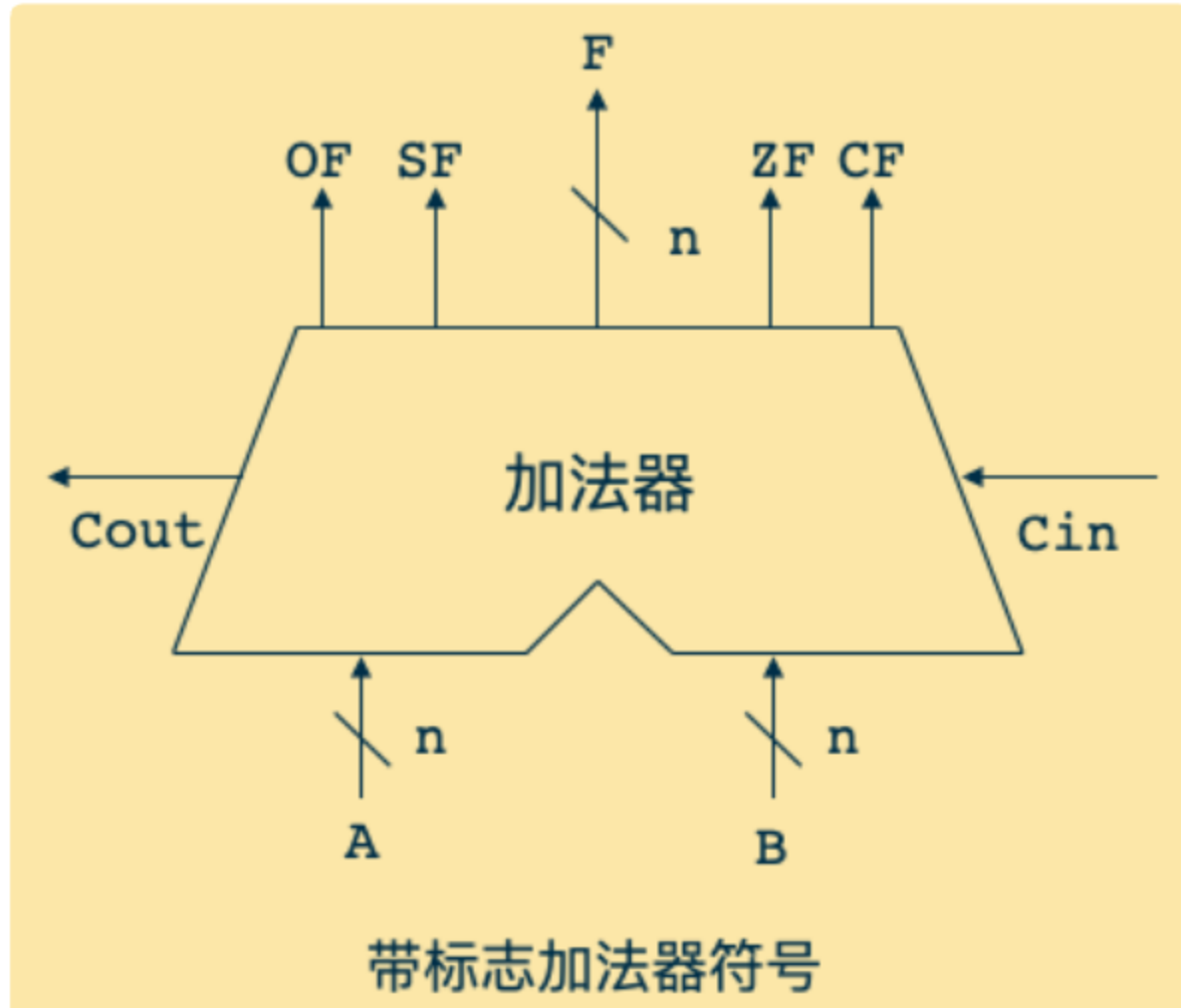
$C_2 = G_2 + P_2 G_1 + P_2 P_1 C_0$

$C_3 = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 C_0$

$C_4 = G_4 + P_4 G_3 + P_4 P_3 G_2 + P_4 P_3 P_2 G_1 + P_4 P_3 P_2 P_1 C_0$

C_i只与低位的X、Y以及初始进位C₀有关, 即, 相互间的进位之间是没有依赖关系的, 只要X₁~X₄, Y₁~Y₄和C₀同时到达, 就可几乎同时形成C₁~C₄, 并同时生成各位的和

符号



概念

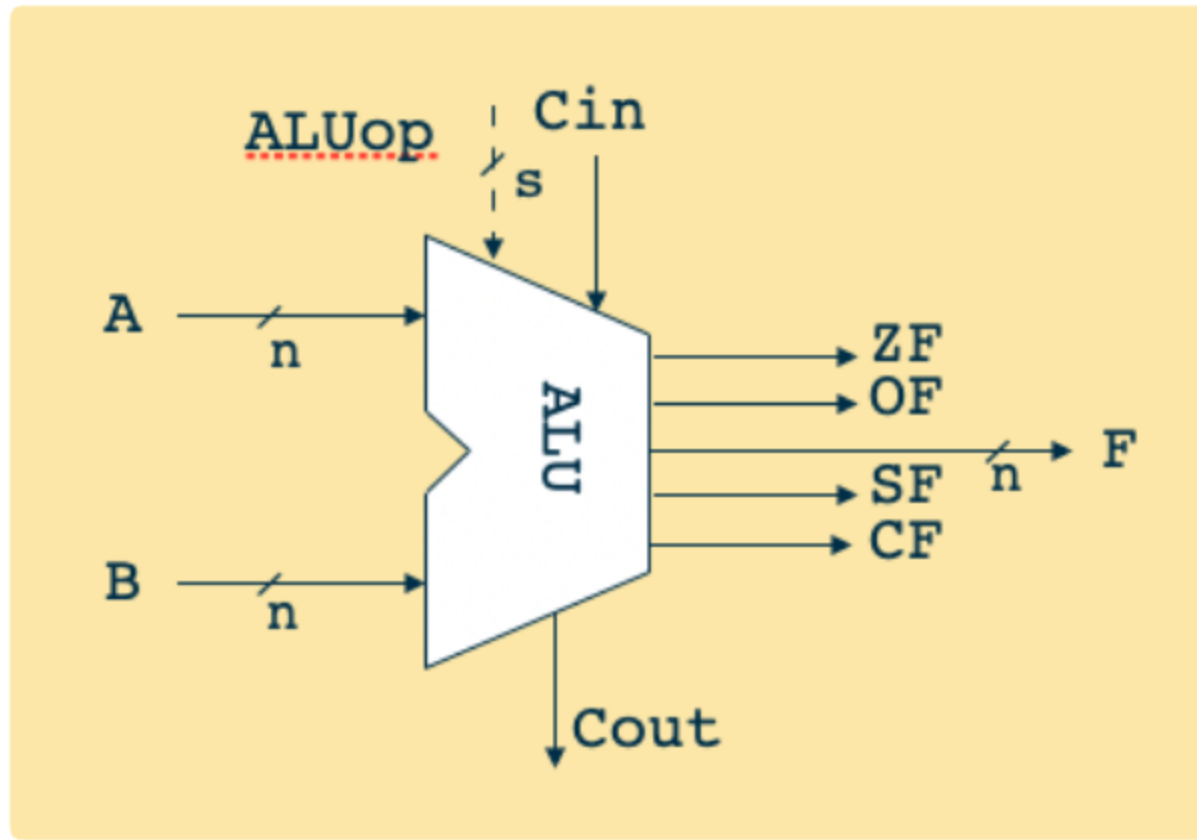
溢出标志 $OF = C_n \oplus C_{n-1}$

符号标志 $SF = F_{n-1}$

零标志 $ZF = 1$ 当且仅当 $F = 0$

进位/借位标志 $CF = Cout \oplus Cin$

符号



2.1移位运算

逻辑移位 (用于无符号数)

左移: 高位移出, 低位补0

右移: 低位移出, 高位补0

算数移位 (用于带符号数 (补码))

左移: 高位移出, 低位补0

右移: 低位移出, 高位补符号位

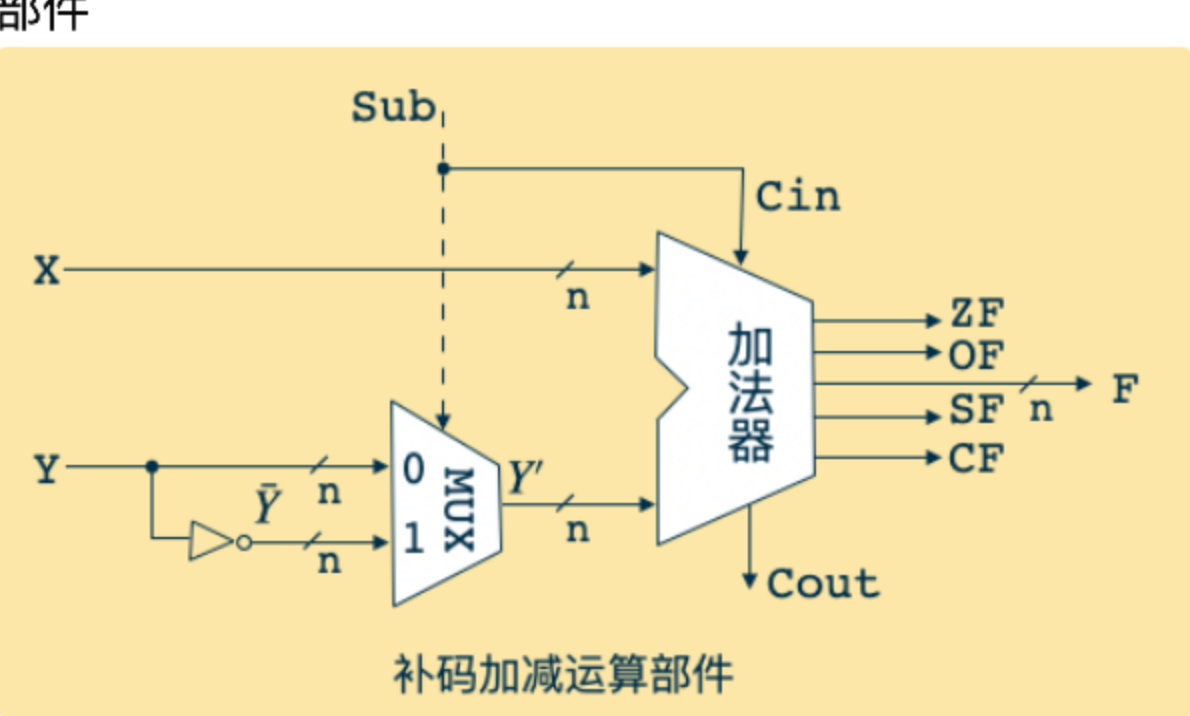
运算法则

每左移1位, 相当于乘2

每右移1位, 相当于除以2

2.2补码加减运算

补码加减运算电路



$[x]_{补} + [y]_{补} = [x+y]_{补}$

$[x]_{补} - [y]_{补} = [x]_{补} + [-y]_{补}$

概念

溢出标志 $OF = C_n \oplus C_{n-1}$

符号标志 $SF = F_{n-1}$

零标志 $ZF = 1$ 当且仅当 $F = 0$

进位/借位标志 $CF = Cout \oplus Cin$

可通过标志信息区分带符号整数运算结果和无符号整数运算结果

SF、OF仅对带符号数有意义, CF仅对无符号数有意义

cmp A,B	无符号数看ZF和CF	若A=B, 则A-B后ZF=1, CF=0
		若A>B, 则A-B后ZF=0, CF=0
		若A<B, 则A-B后ZF=0, CF=1
	带符号数看SF、OF和ZF	若A=B, 则A-B后ZF=1
		若A>B, 则A-B后ZF=0, 且SF⊙OF=1
		若A<B, 则A-B后ZF=0, 且SF⊙OF=0

溢出判断

当且仅当下列4种情况才会发生溢出:

- 正数+正数=负数
- 负数+负数=正数
- 正数-负数=负数
- 负数-正数=正数

采用一位符号位

$V = A_5 B_5 \overline{S_5} + \overline{A_5} B_5 S_5$

V=0, 无溢出

V=1, 有溢出

采用双位符号位

- 设置两个符号位Ss1和Ss2, 最高位Ss1代表真正的符号
- Ss1Ss2 = 00: 结果为正数, 无溢出
- Ss1Ss2 = 01: 结果为正溢出
- Ss1Ss2 = 10: 结果为负溢出
- Ss1Ss2 = 11: 结果为负数, 无溢出

根据数据位的进位情况判断

- 若符号位进位Cs与最高数位进位C1相同, 则无溢出
- 若符号位进位Cs与最高数位进位C1不同, 则发生溢出

2.定点数的运算

2.3原码乘法运算

原码一位乘法

用乘数Y的每一位依次去乘以被乘数得: (i=4,3,2,1), 若Yi=0, 则得0; 若Yi=1, 则得X

把1中求得的各项结果i在空间上向左错位排列, 即逐次左移, 可表示为i

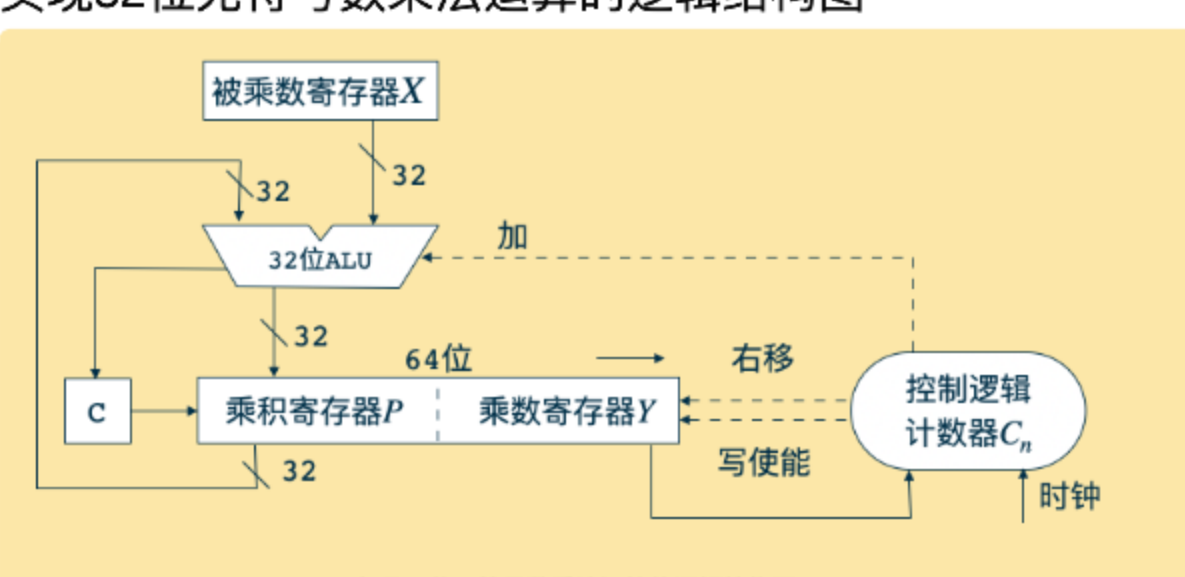
对2中求得的结果求和, 就是两个无符号数的乘积

每将乘数Y的一位乘以被乘数得X*Yi后, 就将该结果与前面所得的结果累加, 得到Pi (部分积)

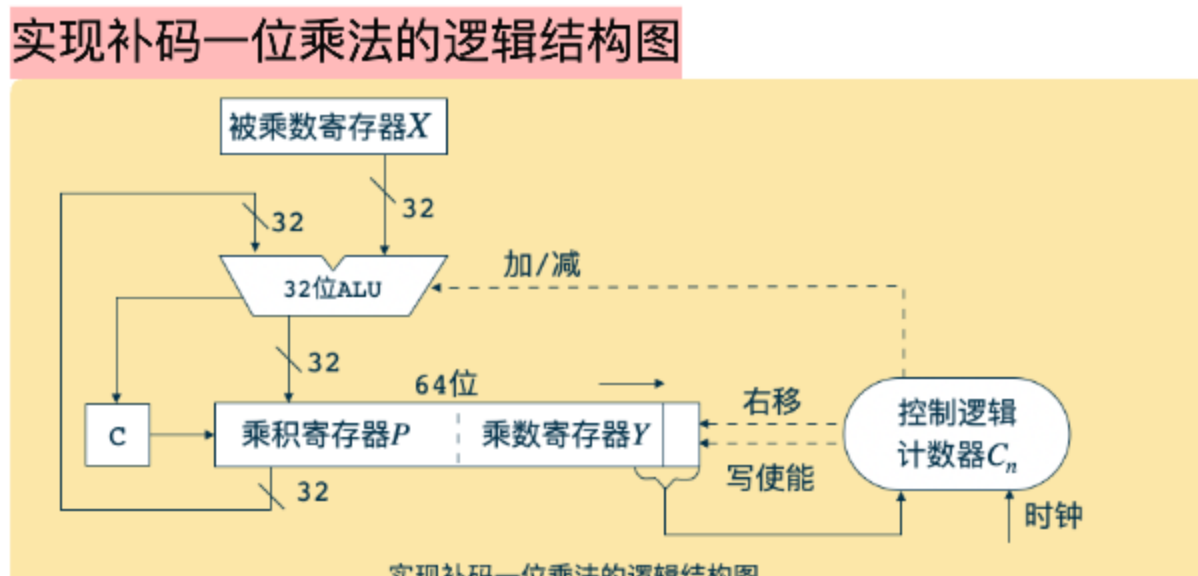
在每次求得X*Yi后, 不是将它左移与前次部分积Pi相加, 而是将部分积Pi右移一位与i相加

对乘数中为1的位执行加法和右移运算, 对为0的位只执行右移运算, 而不需执行加法运算

实现32位无符号数乘法运算的逻辑结构图



实现补码一位乘法的逻辑结构图

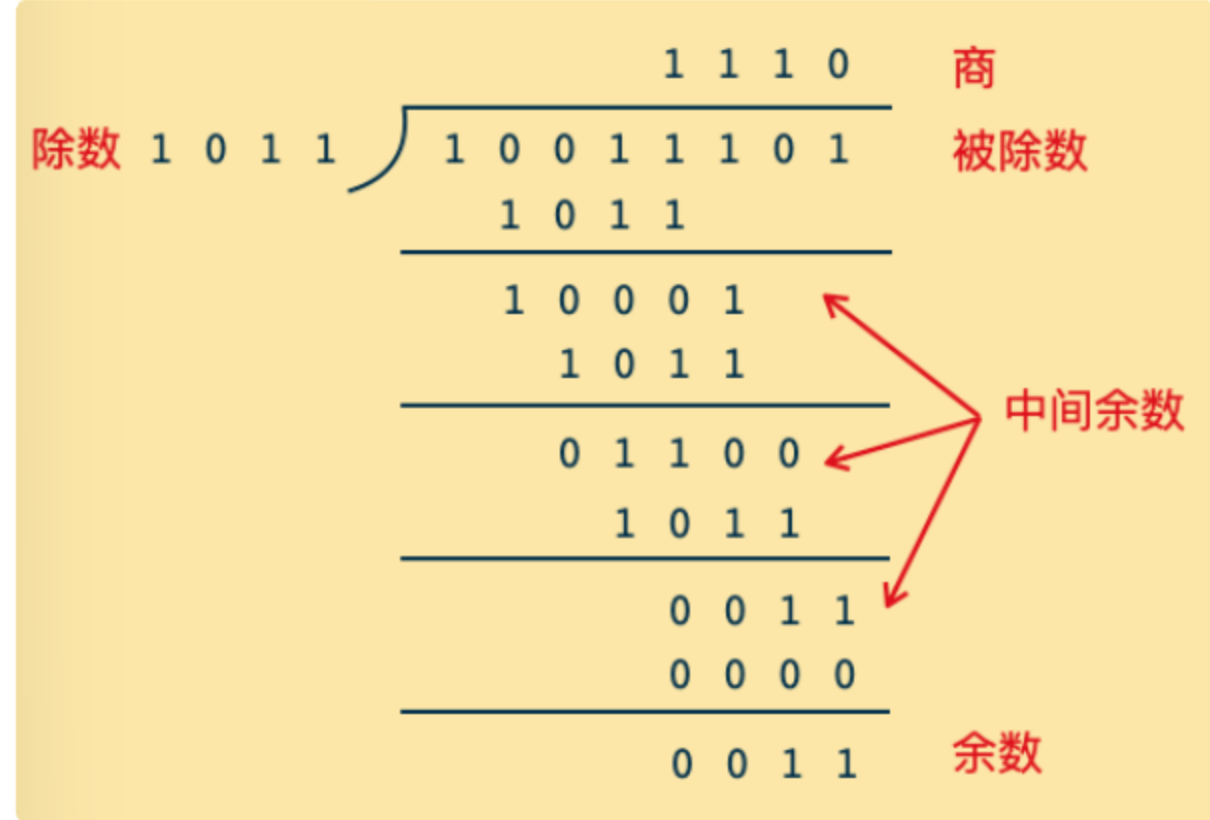


运算法则

- 乘数最低位增加一位辅助位Y-1=0
- 根据YiYi-1的值, 决定是+[X]补、-|X|补还是+0
- 每次加减后, 算数右移1位, 得到部分积
- 重复 (2) (3) n次, 得到结果

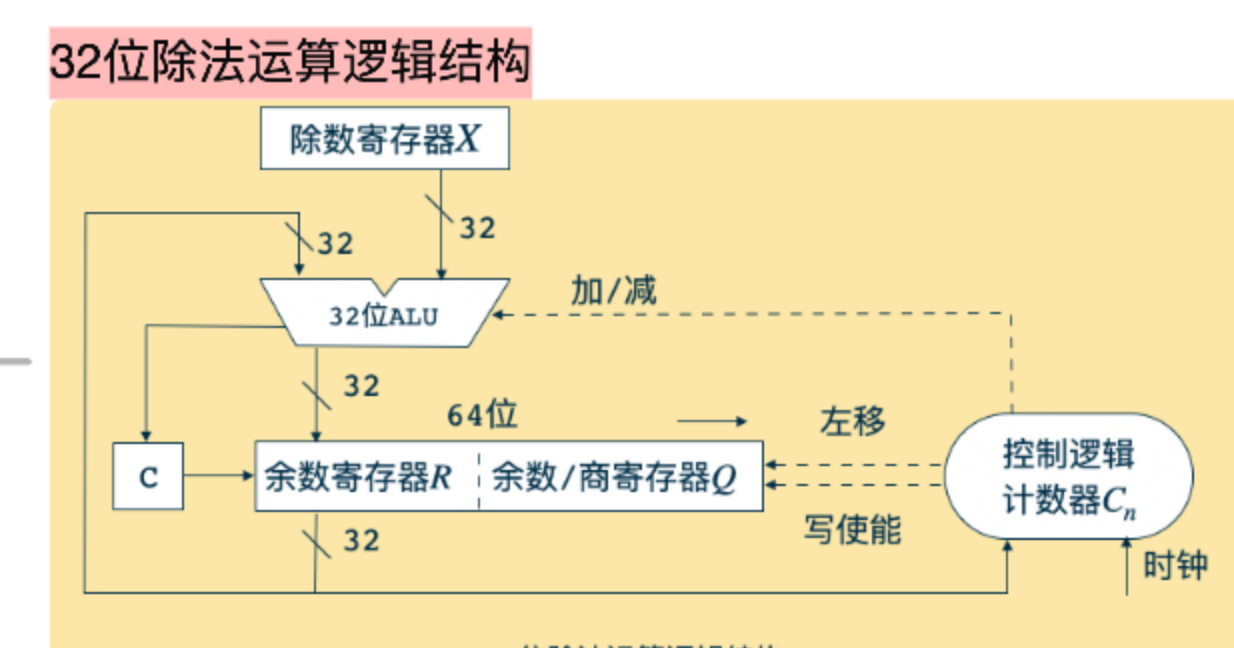
YiYi-1	操作
00	0
01	+[X]补
10	+[-X]补
11	0

2.5原码除法运算



运算法则

- 被除数和除数相减, 若够减, 则上商为1, 否则上0
- 每次得到的差为中间余数, 将除数右移后与上次的中间余数比较, 若够减, 则上商为1, 否则上0
- 重复执行 (2), 直到求得商的位数足够为止



- 初始化: R/Q放被除数, y放除数
- 重复n次:
 - (1) R和Q整体左移1位
 - (2) R=R-Y
- 如果R>=0: 商1
- 如果R<0: 商0, R=R+Y
- 最后:
 - Q中是商
 - R中是余数
- 2n位被除数 ÷ n位除数 → n位商 + n位余数
 - 两个n位定点正整数相除 —— 高位补0
 - 两个n位定点正小数相除 —— 低位补0